

Python

Complete Study Notes

Based on W3Schools Python Tutorial

Introduction • Syntax • Variables • Data Types • Numbers • Casting
Strings • Booleans • Operators • Lists • Tuples • Sets
Dictionaries • If-Else • While Loops • For Loops • Functions
Lambda • Arrays • Classes & Objects • Inheritance • Modules
File Handling • Exception Handling • Iterators • Scope • Date & Time

Comprehensive • Detailed • Example-Based

Table of Contents

Chapter 1. Python Introduction
Chapter 2. Python Syntax & Indentation
Chapter 3. Python Comments
Chapter 4. Python Variables
Chapter 5. Python Data Types
Chapter 6. Python Numbers
Chapter 7. Python Casting (Type Conversion)
Chapter 8. Python Strings
Chapter 9. Python Booleans
Chapter 10. Python Operators
Chapter 11. Python Lists
Chapter 12. Python Tuples
Chapter 13. Python Sets
Chapter 14. Python Dictionaries
Chapter 15. Python If...Else (Conditions)
Chapter 16. Python While Loops
Chapter 17. Python For Loops
Chapter 18. Python Functions
Chapter 19. Python Lambda Functions
Chapter 20. Python Arrays
Chapter 21. Python Classes & Objects
Chapter 22. Python Inheritance
Chapter 23. Python Iterators
Chapter 24. Python Scope
Chapter 25. Python Modules
Chapter 26. Python Dates
Chapter 27. Python File Handling
Chapter 28. Python Exception Handling
Chapter 29. Quick Reference Card

Chapter 1: Python Introduction

What is Python?

Python is a high-level, interpreted, general-purpose programming language. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and simplicity, making it one of the most popular programming languages in the world. The name 'Python' was inspired by the BBC TV show 'Monty Python's Flying Circus'. Python's design philosophy prioritizes developer productivity and code clarity over raw performance.

Python is dynamically typed, which means you do not need to declare the type of a variable before using it. It supports multiple programming paradigms including procedural, object-oriented, and functional programming.

What Can Python Do?

- **Web Development (server-side):** Python frameworks like Django and Flask are widely used to build web applications, APIs, and backend services.
- **Data Science & Machine Learning:** Python is the dominant language in data science, with libraries like NumPy, Pandas, Matplotlib, Scikit-learn, TensorFlow, and PyTorch.
- **Automation & Scripting:** Python automates repetitive tasks such as file operations, web scraping, sending emails, and renaming files.
- **Software Development:** Python can be used to build desktop applications, games, and command-line tools.
- **Database Interaction:** Python can connect to databases like MySQL, PostgreSQL, SQLite, and MongoDB to read, write, and update records.
- **Scientific & Mathematical Computing:** Python is used in academia for simulations, graph plotting, and solving mathematical problems.
- **Cybersecurity & Penetration Testing:** Python is widely used in security tools for network scanning and vulnerability testing.
- **Internet of Things (IoT):** Python runs on Raspberry Pi and other microcontrollers to control hardware.

Why Python is So Popular?

- **Simple and Readable Syntax:** Python syntax is very similar to the English language, making it easy to learn and understand, even for complete beginners.
- **Fewer Lines of Code:** Tasks that require 10 lines in Java or C++ can often be done in 3-5 lines in Python.
- **Large Standard Library:** Python comes with a rich standard library that covers string operations, web protocols, file I/O, data serialization, and much more.
- **Huge Community & Ecosystem:** Python has millions of developers worldwide and thousands of third-party libraries available via PyPI.

- Cross-Platform: Python programs run on Windows, Linux, macOS, and even mobile devices without modification.
- Interpreted Language: Python code is executed line by line, making debugging easy and allowing you to test code interactively.
- Free & Open Source: Python is completely free to download and use, even for commercial applications.

Your First Python Program

The traditional first program in any language is 'Hello, World!'. In Python, this is just one line:

```
print("Hello, world!")
```

Output:

```
Hello, World!
```

■ *Note: The `print()` function displays output to the screen. You can pass any text (string) or variable to it. Python does NOT require a semicolon at the end of statements — a new line is enough.*

Chapter 2: Python Syntax & Indentation

Python Syntax

Python syntax refers to the set of rules that define how a Python program is written and interpreted. Unlike many other programming languages, Python does not use curly braces `{}` to define blocks of code. Instead, Python uses indentation (whitespace at the beginning of lines) to determine the structure of the code.

Python is case-sensitive, which means 'Variable', 'variable', and 'VARIABLE' are three completely different identifiers. Python statements are generally written one per line. If you need to continue a statement on the next line, you can use the backslash (`\`) continuation character or wrap the statement in parentheses.

Indentation — The Heart of Python

Indentation is not optional in Python — it is mandatory. In most other languages, indentation is used only for readability. In Python, incorrect indentation will cause an `IndentationError` and your program will not run.

The standard indentation is 4 spaces. You can use any consistent number of spaces (or a tab), but you must be consistent throughout the same block.

```
# Correct indentation

if 5 > 2:

    print("Five is greater than two!")

    print("This is also inside the if block")


# This would cause an IndentationError

if 5 > 2:

    print("Two spaces")

    print("Four spaces – IndentationError!") # Inconsistent!
```

Output:

```
Five is greater than two!

This is also inside the if block
```

Python Statements

In Python, the end of a statement is marked by a newline character. However, you can use a semicolon to write multiple statements on a single line (though this is generally discouraged for readability):

```
x = 5; y = 10; z = 15 # Multiple statements on one line (not recommended)

# Multi-line statement using backslash
total = 1 + 2 + 3 + \
4 + 5 + 6
print(total)

# Multi-line using parentheses (preferred)
total = (1 + 2 + 3 +
4 + 5 + 6)
print(total)
```

Output:

```
21
21
```

Python Identifiers

An identifier is a name given to a variable, function, class, or module. Rules for identifiers in Python:

- Must begin with a letter (a-z, A-Z) or an underscore (_).
- Cannot start with a digit.
- Can only contain letters, digits, and underscores.
- Cannot be a Python keyword (like if, else, while, for, class, etc.).
- Are case-sensitive (myVar and myvar are different).

```
# Valid identifiers

name = 'Alice'

_age = 25

user1 = 'Bob'

MyClass = 'something'
```

```
# Invalid identifiers (these would cause errors)
```

```
# 1name = 'Invalid' # starts with digit
```

```
# my-name = 'Invalid' # contains hyphen
```

```
# class = 'Invalid' # reserved keyword
```

Chapter 3: Python Comments

What are Comments?

Comments are lines in your code that Python ignores during execution. They are used to explain what the code does, leave notes for yourself or other developers, and temporarily disable a section of code during debugging. Good commenting practices make code easier to understand and maintain.

Single-Line Comments

A single-line comment starts with a hash symbol (#). Everything after the # on that line is treated as a comment and is completely ignored by Python. Comments can appear on their own line or at the end of a code line.

```
# This is a standalone comment

print("Hello, World!") # This is an inline comment

# You can explain what the next line does

x = 5 # assign the value 5 to variable x

y = x + 10 # add 10 to x and store result in y

print(y)
```

Output:

```
Hello, World!

15
```

Multi-Line Comments

Python does not have a dedicated multi-line comment syntax like `/* ... */` in C or Java. However, there are two common ways to write multi-line comments in Python:

Method 1: Multiple # symbols

```
# This is line 1 of a multi-line comment

# This is line 2 of a multi-line comment

# This is line 3 of a multi-line comment

print("Code continues here")
```

Method 2: Triple-quoted strings (Docstrings)


```
"""

This is a multi-line comment using

triple double-quotes. Python treats

unassigned strings as no-operation.

"""

'''

This also works with

triple single-quotes.

'''

print("Code after the comment block")
```

■ *Note: Triple-quoted strings are officially called 'docstrings' when used at the start of a function, class, or module. They serve as documentation and can be accessed via the `__doc__` attribute. When used elsewhere, they act as multi-line comments since they are string literals that are never assigned to any variable.*

Chapter 4: Python Variables

What is a Variable?

A variable is a named storage location in the computer's memory that holds a value. Variables allow you to store data and refer to it by name throughout your program. In Python, you do NOT need to declare the type of a variable before using it. A variable is created automatically the moment you first assign a value to it using the assignment operator (=).

Python is dynamically typed, meaning the type of a variable is determined at runtime and can change during the program's execution. This is a major difference from statically typed languages like C, C++, or Java where you must specify the type (int, string, float) before using a variable.

```
x = 5 # x is an integer

y = "John" # y is a string

z = 3.14 # z is a float

print(x) # prints 5

print(y) # prints John

print(z) # prints 3.14
```

Output:

```
5

John

3.14
```

Variable Naming Rules

- A variable name must start with a letter (a-z or A-Z) or an underscore (_).
- A variable name cannot start with a number.
- Variable names can only contain alphanumeric characters and underscores (A-z, 0-9, _).
- Variable names are case-sensitive: age, Age, and AGE are three entirely different variables.
- Variable names cannot be Python reserved keywords like if, else, while, for, True, False, None, etc.
- Variable names should be descriptive and meaningful (use student_name instead of sn).

```
# Valid variable names

myvar = 'hello'
```

```
my_var = 'hello'

_myvar = 'hello'

myVar = 'hello'

MYVAR = 'hello'

myvar2 = 'hello'


# Invalid variable names – these cause SyntaxError

# 2myvar = 'hello' # starts with a number

# my-var = 'hello' # contains a hyphen

# my var = 'hello' # contains a space
```

Assigning Multiple Values

Assign Multiple Variables in One Line

Python allows you to assign values to multiple variables in a single line — a feature called multiple assignment or tuple unpacking:

```
x, y, z = "Orange", "Banana", "Cherry"

print(x) # Orange

print(y) # Banana

print(z) # Cherry
```

Output:

```
Orange
Banana
Cherry
```

Assign the Same Value to Multiple Variables

```
x = y = z = "Python"

print(x) # Python

print(y) # Python

print(z) # Python
```

Get the Type of a Variable

You can check the type of any variable using the built-in `type()` function. This is especially useful when debugging or when you are not sure what type a variable holds:

```
x = 5

y = "Hello"

z = 3.14

a = True

b = [1, 2, 3]

print(type(x)) # int

print(type(y)) # str

print(type(z)) # float

print(type(a)) # bool

print(type(b)) # list
```

Output:

Global vs Local Variables

Variables created outside of a function are called global variables. They can be accessed from anywhere in the program. Variables created inside a function are called local variables. They only exist within that function and cannot be accessed from outside.

```
x = "global" # Global variable

def my_function():

    y = "local" # Local variable

    print(x) # Can access global variable

    print(y) # Can access local variable

my_function()
```

```
print(x) # Works fine
```

```
# print(y) # Would cause NameError: y is not defined
```

The global Keyword

If you need to modify a global variable from inside a function, use the global keyword:

```
x = "awesome"
```

```
def my_function():
```

```
    global x
```

```
    x = "fantastic"
```

```
my_function()
```

```
print(x) # fantastic (modified by the function)
```

Output:

```
fantastic
```

Chapter 5: Python Data Types

Understanding Data Types

In programming, a data type defines the kind of value a variable can hold and what operations can be performed on it. Python has a rich set of built-in data types. Since Python is dynamically typed, it automatically assigns the correct data type when you create a variable — you don't have to specify it manually. Understanding data types is crucial because it determines how data is stored in memory and what you can do with it.

Category	Data Types	Example
Text Type	str	"Hello"
Numeric Types	int, float, complex	5, 3.14, 2+3j
Sequence Types	list, tuple, range	[1,2,3], (1,2), range(5)
Mapping Type	dict	{"name": "Alice"}
Set Types	set, frozenset	{1, 2, 3}
Boolean Type	bool	True, False
Binary Types	bytes, bytearray, memoryview	b'hello'
None Type	NoneType	None

Getting the Data Type

Use the `type()` function to check the data type of any variable at runtime:

```
print(type(5)) # int

print(type(3.14)) # float

print(type(2+3j)) # complex

print(type("Hello")) # str

print(type([1, 2, 3])) # list

print(type((1, 2, 3))) # tuple

print(type({1, 2, 3})) # set

print(type({"a": 1})) # dict

print(type(True)) # bool

print(type(None)) # NoneType
```

Setting Specific Data Types

You can explicitly set a specific data type by using Python's built-in constructor functions. This is called type casting or type conversion:

```
x = str('Hello') # str

x = int(20) # int

x = float(20.5) # float

x = complex(1j) # complex

x = list(('a','b','c')) # list

x = tuple(('a','b','c')) # tuple

x = range(6) # range

x = dict(name='John') # dict

x = set(('a','b','c')) # set

x = frozenset(('a','b')) # frozenset

x = bool(5) # bool

x = bytes(5) # bytes

x = bytearray(5) # bytearray
```

Chapter 6: Python Numbers

Python Number Types

Python supports three distinct numeric types: integers (int), floating-point numbers (float), and complex numbers (complex). Each serves different purposes in programming, and Python automatically selects the appropriate type based on the value you assign.

Integers (int)

An integer is a whole number — positive, negative, or zero — without any decimal point. In Python, integers have unlimited precision, meaning they can be as large as your computer's memory allows. There is no fixed maximum size for integers in Python (unlike C or Java).

```
x = 1

y = 35656222554887711

z = -3255522

big_num = 123456789012345678901234567890 # No size limit!

print(type(x)) # <class 'int'>

print(big_num) # Python handles it perfectly
```

Output:

```
123456789012345678901234567890
```

Floating Point Numbers (float)

Floating point numbers (floats) are numbers that contain a decimal point. They can be positive or negative. Python floats are 64-bit double-precision numbers following the IEEE 754 standard, which provides about 15-17 decimal digits of precision. You can also use scientific notation with the letter 'e' to represent large or very small floats.

```
x = 1.10

y = 1.0

z = -35.59

# Scientific notation
```



```
a = 35e3 # 35000.0

b = 12E4 # 120000.0

c = -87.7e100 # extremely large number

print(type(x)) # <class 'float'>

print(a) # 35000.0

print(b) # 120000.0
```

Output:

```
35000.0

120000.0
```

Complex Numbers (complex)

Complex numbers have a real part and an imaginary part. In Python, the imaginary part is denoted by the letter 'j' (not 'i' as in mathematics). Complex numbers are used in scientific computing, signal processing, electrical engineering, and other advanced fields.

```
x = 3 + 5j

y = 5j

z = -5j

print(type(x)) # <class 'complex'>

print(x.real) # 3.0 (real part)

print(x.imag) # 5.0 (imaginary part)

print(x + y) # (3+10j)
```

Output:

```
3.0

5.0

(3+10j)
```

Type Conversion Between Numbers

You can convert between numeric types using `int()`, `float()`, and `complex()` functions. Note: you cannot convert a complex number directly to `int` or `float`.

```
# int to float

x = float(5) # 5.0

# float to int (truncates, does NOT round)

y = int(2.9) # 2 (NOT 3!)

# int to complex

z = complex(5) # (5+0j)

print(x, y, z)
```

Output:

```
5.0 2 (5+0j)
```

Random Numbers

Python does not have a built-in random number generator, but the `random` module from the standard library provides many functions for generating random numbers. This is useful for simulations, games, testing, and security applications.

```
import random

print(random.randrange(1, 10)) # random int from 1 to 9

print(random.random()) # random float between 0.0 and 1.0

print(random.randint(1, 100)) # random int from 1 to 100 (inclusive)

# Random choice from a list

fruits = ["apple", "banana", "cherry"]

print(random.choice(fruits)) # random item from list
```

Chapter 7: Python Casting (Type Conversion)

What is Casting?

Casting (also called type conversion) is the process of converting a variable from one data type to another. This is often necessary when you receive data in one format (e.g., as a string from user input) but need it in another format (e.g., as an integer for calculations). Python provides built-in functions for all common type conversions.

There are two kinds of type conversion in Python: Implicit conversion (Python does it automatically) and Explicit conversion (you do it manually using built-in functions). When there is no risk of data loss, Python converts types automatically. When there is a risk, you must do it explicitly.

Implicit Type Conversion

Python automatically converts a smaller data type to a larger one to prevent data loss. For example, when you add an integer and a float, Python automatically converts the integer to float:

```
num_int = 123 # int

num_flo = 1.23 # float

num_new = num_int + num_flo

print(num_new) # 124.23

print(type(num_new)) # float (Python promoted int to float automatically)
```

Output:

```
124.23
```

Explicit Type Conversion

Converting to Integer — int()

The int() function converts a float, string (if it represents a whole number), or boolean to an integer. When converting a float to int, it truncates (removes) the decimal part — it does NOT round.

```
x = int(1) # 1 (int to int – no change)

y = int(2.8) # 2 (float to int – truncated, NOT rounded!)

z = int("3") # 3 (string to int)

a = int(True) # 1 (bool to int)
```

```
b = int(False) # 0 (bool to int)

print(x, y, z, a, b)

# This would raise a ValueError:

# int("3.14") # Cannot convert decimal string directly
```

Output:

```
1 2 3 1 0
```

Converting to Float — float()

The float() function converts integers, numeric strings, and booleans to floating-point numbers:

```
x = float(1) # 1.0

y = float(2.8) # 2.8

z = float("3") # 3.0

w = float("4.2") # 4.2

v = float(True) # 1.0

print(x, y, z, w, v)
```

Output:

```
1.0 2.8 3.0 4.2 1.0
```

Converting to String — str()

The str() function converts any data type to its string representation:

```
x = str(3) # "3"

y = str(3.0) # "3.0"

z = str(True) # "True"

a = str([1,2,3]) # "[1, 2, 3]"

print(type(x)) # <class 'str'>

# Useful for concatenation:
```

```
age = 25

message = "I am " + str(age) + " years old"

print(message)
```

Output:

```
I am 25 years old
```

■ *Important: `int('3.14')` raises a `ValueError`. To convert a decimal string to int, first convert to float: `int(float('3.14'))` gives 3.*

Chapter 8: Python Strings

What are Strings?

A string is a sequence of characters enclosed within single quotes (' '), double quotes (" "), or triple quotes (''' ' or '''' '''). Strings are one of the most commonly used data types in Python. They are immutable, meaning once a string is created, you cannot modify individual characters within it — you can only create new strings.

```
a = "Hello World" # double quotes

b = 'Hello World' # single quotes – both are the same

c = """This is a
multiline string""" # triple quotes for multi-line

print(a)

print(type(a)) # <class 'str'>
```

Output:

```
Hello World
```

Accessing Characters — Indexing

Each character in a string has an index (position). Python uses zero-based indexing, meaning the first character is at index 0. You can also use negative indexing where -1 refers to the last character, -2 to the second last, and so on.

```
a = "Hello, World!"

# Positive indexing

print(a[0]) # H (first character)

print(a[1]) # e

print(a[7]) # W

# Negative indexing

print(a[-1]) # ! (last character)
```

```
print(a[-5]) # o
```

Output:

```
H
e
W
!
o
```

String Slicing

Slicing allows you to extract a portion of a string. The syntax is `string[start:end:step]`. The start index is inclusive and the end index is exclusive. If you omit start, slicing begins from the start. If you omit end, slicing goes to the end of the string.

```
b = "Hello, World!"

print(b[2:5]) # llo (index 2, 3, 4 – NOT including 5)

print(b[:5]) # Hello (from start to index 4)

print(b[7:]) # World! (from index 7 to end)

print(b[-5:-2]) # orl (negative slicing)

print(b[::2]) # Hlo ol! (every 2nd character – step)

print(b[::-1]) # !dlrow ,olleH (reverse the string)
```

Output:

```
llo
Hello
World!
orl
Hlo ol!
!dlrow ,olleH
```

String Length

The `len()` function returns the total number of characters in a string, including spaces and punctuation:

```
a = "Hello, world!"

print(len(a)) # 13
```

Output:

13

Important String Methods

Python provides many powerful built-in string methods. They all return a new string (strings are immutable):

Method	Description	Example
<code>upper()</code>	Converts all characters to uppercase	<code>"hello".upper()</code> → <code>"HELLO"</code>
<code>lower()</code>	Converts all characters to lowercase	<code>"HELLO".lower()</code> → <code>"hello"</code>
<code>strip()</code>	Removes leading & trailing whitespace (or specified characters)	<code>" Hi ".strip()</code> → <code>"Hi"</code>
<code>lstrip()</code>	Removes leading (left) whitespace	<code>" Hi ".lstrip()</code> → <code>"Hi "</code>
<code>rstrip()</code>	Removes trailing (right) whitespace	<code>" Hi ".rstrip()</code> → <code>" Hi"</code>
<code>replace(a, b)</code>	Replaces all occurrences of a with b	<code>"Hi".replace("H","J")</code> → <code>"Ji"</code>
<code>split(sep)</code>	Splits string into a list at separator	<code>"a,b,c".split(",")</code> → <code>["a","b","c"]</code>
<code>join(iterable)</code>	Joins a list into a string with separator	<code>"-".join(["a","b"])</code> → <code>"a-b"</code>
<code>find(sub)</code>	Returns index of first occurrence (-1 if not found)	<code>"hello".find("l")</code> → <code>2</code>
<code>count(sub)</code>	Counts occurrences of a substring	<code>"banana".count("a")</code> → <code>3</code>
<code>startswith(s)</code>	Returns True if string starts with s	<code>"Hello".startswith("He")</code> → <code>True</code>
<code>endswith(s)</code>	Returns True if string ends with s	<code>"Hello".endswith("lo")</code> → <code>True</code>
<code>isalpha()</code>	True if all characters are alphabetic	<code>"Hello".isalpha()</code> → <code>True</code>
<code>isdigit()</code>	True if all characters are digits	<code>"123".isdigit()</code> → <code>True</code>
<code>isspace()</code>	True if all characters are whitespace	<code>" ".isspace()</code> → <code>True</code>
<code>title()</code>	Converts first letter of each word to uppercase	<code>"hello world".title()</code> → <code>"Hello World"</code>
<code>capitalize()</code>	Capitalizes first letter only	<code>"hello".capitalize()</code> → <code>"Hello"</code>
<code>center(n)</code>	Centers string in a field of width n	<code>"hi".center(10)</code> → <code>" hi "</code>
<code>zfill(n)</code>	Pads string on the left with zeros	<code>"42".zfill(5)</code> → <code>"00042"</code>

String Formatting

f-Strings (Recommended — Python 3.6+)

f-Strings (formatted string literals) are the modern and preferred way to embed expressions inside strings. Prefix the string with 'f' and place variables or expressions inside curly braces {}.


```
name = 'John'

age = 36

price = 19.99

txt = f"My name is {name}, I am {age} years old."

print(txt)

# You can put expressions inside {}

print(f"Next year I will be {age + 1}")

# Formatting numbers

print(f"Price: ${price:.2f}") # 2 decimal places
```

Output:

```
My name is John, I am 36 years old.

Next year I will be 37

Price: $19.99
```

format() Method

```
txt = "For only {price:.2f} dollars!"

print(txt.format(price=49))

# Positional arguments

print("{0} is {1} years old".format("Alice", 30))
```

Output:

```
For only 49.00 dollars!

Alice is 30 years old
```

String Concatenation and Repetition

```
# Concatenation (joining strings)

a = "Hello"
```

```
b = " World"

c = a + b

print(c) # Hello World


# Repetition

print("Ha" * 3) # HaHaHa


# Concatenate with a non-string requires str()

age = 25

print("I am " + str(age)) # I am 25
```

Chapter 9: Python Booleans

What are Booleans?

A Boolean represents one of only two possible values: True or False. Booleans are named after the mathematician George Boole, who developed Boolean algebra. In Python, the Boolean data type is `bool` and is a subclass of `int`. True is equivalent to 1 and False is equivalent to 0 in arithmetic operations.

Booleans are essential in programming because they are the foundation of all conditional logic (if/else statements), loop conditions, and logical operations. Most comparison operations return Boolean values.

Boolean Values

```
x = True

y = False

print(type(x)) # <class 'bool'>

# Booleans are subclasses of int

print(True + True) # 2

print(True + False) # 1

print(True * 5) # 5

print(False * 5) # 0
```

Output:

```
2
1
5
0
```

Evaluating Values and Variables

The `bool()` function converts any value to a Boolean. This is useful to understand which values Python considers as 'truthy' (evaluates to True) and which are 'falsy' (evaluates to False). Almost all values are True except for a specific set of values:

Values that evaluate to FALSE (Falsy values):

- The number 0 (integer zero)
- The float 0.0
- The complex number 0j
- An empty string: ""
- An empty list: []
- An empty tuple: ()
- An empty dictionary: {}
- An empty set: set()
- The None object
- The boolean False itself

```
print(bool('Hello')) # True (non-empty string)

print(bool(15)) # True (non-zero int)

print(bool([1,2])) # True (non-empty list)


print(bool('')) # False (empty string)

print(bool(0)) # False (zero)

print(bool([])) # False (empty list)

print(bool(None)) # False
```

Output:

```
True
True
True
False
False
False
False
```

Comparison Operators Return Booleans

```
print(10 > 9) # True

print(10 == 9) # False
```

```
print(10 < 9) # False

# Used in if statements

a = 200

b = 33

if b > a:

    print("b is greater than a")

else:

    print("a is greater than b")
```

Output:

True

False

False

a is greater than b

Chapter 10: Python Operators

What are Operators?

Operators are special symbols or keywords that perform operations on variables and values. The values that operators act on are called operands. Python has a rich set of operators grouped into several categories: Arithmetic, Assignment, Comparison, Logical, Identity, Membership, and Bitwise operators.

1. Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations:

Operator	Name	Example	Result	Explanation
+	Addition	5 + 2	7	Adds two operands
-	Subtraction	5 - 2	3	Subtracts right from left
*	Multiplication	5 * 2	10	Multiplies two operands
/	Division	5 / 2	2.5	Always returns float
%	Modulus	5 % 2	1	Returns division remainder
**	Exponentiation	5 ** 2	25	Raises left to power of right
//	Floor Division	5 // 2	2	Division rounded down to integer

2. Assignment Operators

Assignment operators are used to assign values to variables. The compound operators perform an operation and assign in one step:

Operator	Example	Equivalent To	Description
=	x = 5	x = 5	Simple assignment
+=	x += 3	x = x + 3	Add and assign
-=	x -= 3	x = x - 3	Subtract and assign
*=	x *= 3	x = x * 3	Multiply and assign
/=	x /= 3	x = x / 3	Divide and assign
%=	x %= 3	x = x % 3	Modulus and assign
**=	x **= 3	x = x ** 3	Exponent and assign
//=	x //= 3	x = x // 3	Floor divide and assign

3. Comparison Operators

Comparison operators compare two values and return a Boolean (True or False) result:

Operator	Name	Example	Result	Explanation
==	Equal	5 == 5	True	True if both values are equal
!=	Not Equal	5 != 3	True	True if values are NOT equal
>	Greater Than	5 > 3	True	True if left is greater
<	Less Than	3 < 5	True	True if left is smaller
>=	Greater or Equal	5 >= 5	True	True if left >= right
<=	Less or Equal	3 <= 5	True	True if left <= right

4. Logical Operators

Logical operators are used to combine conditional statements:

```
x = 5

# 'and' – True only if BOTH conditions are True

print(x > 3 and x < 10) # True (5>3 AND 5<10)

print(x > 3 and x > 10) # False (5>3 is True but 5>10 is False)

# 'or' – True if AT LEAST ONE condition is True

print(x > 3 or x < 4) # True (5>3 is True)

print(x < 3 or x > 10) # False (both are False)

# 'not' – Reverses/negates the result

print(not(x > 3 and x < 10)) # False (negates True)

print(not(x < 3)) # True (negates False)
```

Output:

```
True
False
True
False
False
```

True

5. Identity Operators

Identity operators check whether two variables point to the same object in memory — not whether they have the same value. The 'is' operator checks identity, while '==' checks equality.

```
x = ['apple', 'banana']  
  
y = ['apple', 'banana']  
  
z = x # z points to the same object as x  
  
print(x is z) # True (same object in memory)  
  
print(x is y) # False (same values but DIFFERENT objects)  
  
print(x == y) # True (same values)  
  
print(x is not y) # True (not the same object)
```

Output:

```
True  
  
False  
  
True  
  
True
```

6. Membership Operators

Membership operators test whether a value or variable is found in a sequence (string, list, tuple, set, dict):

```
fruits = ["apple", "banana", "cherry"]  
  
# "in" – True if value exists in the sequence  
  
print("banana" in fruits) # True  
  
print("grape" in fruits) # False  
  
# "not in" – True if value does NOT exist  
  
print("grape" not in fruits) # True
```



```
# Works with strings too:
```

```
print("an" in "banana") # True
```

Output:

```
True
```

```
False
```

```
True
```

```
True
```

Chapter 11: Python Lists

What is a List?

A list is an ordered, mutable (changeable), and indexed collection that can hold items of different data types. Lists are one of the most versatile and frequently used data structures in Python. They are created using square brackets `[]` and items are separated by commas. Lists maintain the order of insertion and allow duplicate values.

```
# A simple list

thislist = ["apple", "banana", "cherry"]

print(thislist) # ['apple', 'banana', 'cherry']

print(len(thislist)) # 3 (number of items)

print(type(thislist)) # <class 'list'>

# Lists can hold mixed data types

mixed = [1, "Hello", 3.14, True, None]

print(mixed)
```

Accessing List Items

Access items by their index (0-based). Negative indexing counts from the end:

```
thislist = ["apple", "banana", "cherry", "date"]

print(thislist[0]) # apple (first item)

print(thislist[2]) # cherry

print(thislist[-1]) # date (last item)

print(thislist[-2]) # cherry (second from end)

# Slicing – get a range of items

print(thislist[1:3]) # ['banana', 'cherry'] (index 1 and 2)

print(thislist[:2]) # ['apple', 'banana'] (from start to index 1)
```

```
print(thislist[2:]) # ['cherry', 'date'] (from index 2 to end)
```

Output:

```
apple
cherry
date
cherry
['banana', 'cherry']
['apple', 'banana']
['cherry', 'date']
```

Modifying List Items

Since lists are mutable, you can change, add, and remove items after creation:

```
# Change a single item

thislist = ["apple", "banana", "cherry"]

thislist[1] = "blackcurrant"

print(thislist) # ['apple', 'blackcurrant', 'cherry']


# Change a range of items

thislist[1:3] = ["melon", "watermelon"]

print(thislist) # ['apple', 'melon', 'watermelon']
```

Adding Items to a List

```
fruits = ["apple", "banana", "cherry"]


# append() – adds to the END

fruits.append("orange")

print(fruits) # ['apple', 'banana', 'cherry', 'orange']


# insert() – adds at a SPECIFIC position
```

```
fruits.insert(1, "mango")

print(fruits) # ['apple', 'mango', 'banana', 'cherry', 'orange']

# extend() – adds elements from another list

more = ["grape", "kiwi"]

fruits.extend(more)

print(fruits)
```

Removing Items from a List

```
fruits = ["apple", "banana", "cherry", "banana"]

# remove() – removes the FIRST occurrence of the value

fruits.remove("banana")

print(fruits) # ['apple', 'cherry', 'banana']

# pop() – removes item at index (default: last item)

fruits.pop() # removes 'banana' (last)

fruits.pop(0) # removes 'apple' (index 0)

print(fruits) # ['cherry']

# del – removes by index or the entire list

lst = [1, 2, 3, 4, 5]

del lst[0] # removes first item

print(lst) # [2, 3, 4, 5]

# clear() – empties the list (list still exists)

lst.clear()

print(lst) # []
```

List Comprehension

List comprehension is a concise and elegant way to create a new list based on an existing list or any iterable. It replaces multi-line for loops with a single, readable expression. Syntax: [expression for item in iterable if condition]

```
# Traditional way (for loop)

fruits = ['apple', 'banana', 'cherry', 'kiwi', 'mango']

newlist = []

for x in fruits:

    if "a" in x:

        newlist.append(x)


# List comprehension – same result in ONE line

newlist = [x for x in fruits if "a" in x]

print(newlist) # ['apple', 'banana', 'mango']


# Create a list of squares

squares = [x**2 for x in range(1, 6)]

print(squares) # [1, 4, 9, 16, 25]


# Uppercase all items

upper = [x.upper() for x in fruits]

print(upper) # ['APPLE', 'BANANA', 'CHERRY', 'KIWI', 'MANGO']
```

Output:

```
['apple', 'banana', 'mango']

[1, 4, 9, 16, 25]

['APPLE', 'BANANA', 'CHERRY', 'KIWI', 'MANGO']
```

Sorting Lists

```
fruits = ["orange", "mango", "kiwi", "pineapple", "banana"]
```

```
fruits.sort() # alphabetical (in-place)

print(fruits)

nums = [100, 50, 65, 82, 23]

nums.sort() # ascending (in-place)

print(nums)

nums.sort(reverse=True) # descending

print(nums)

# sorted() – returns a new list, original unchanged

original = [3, 1, 4, 1, 5]

new = sorted(original)

print(original) # [3, 1, 4, 1, 5]

print(new) # [1, 1, 3, 4, 5]
```

Chapter 12: Python Tuples

What is a Tuple?

A tuple is an ordered, immutable (unchangeable) collection of items. Tuples are very similar to lists, but with one critical difference: once a tuple is created, you cannot add, remove, or change its items. Tuples are created using parentheses () and items are separated by commas. Because tuples are immutable, they are faster than lists and can be used as dictionary keys.

Tuples are commonly used to store related pieces of data that should not change, such as coordinates (x, y), RGB color values (r, g, b), or database records.

```
# Creating a tuple

thistuple = ("apple", "banana", "cherry")

print(thistuple) # ('apple', 'banana', 'cherry')

print(type(thistuple)) # <class 'tuple'>

print(len(thistuple)) # 3


# A tuple with ONE item needs a trailing comma!

single = ("apple",) # WITH comma – this is a tuple

not_tuple = ("apple") # WITHOUT comma – this is just a string!

print(type(single)) # <class 'tuple'>

print(type(not_tuple)) # <class 'str'>
```

Accessing Tuple Items

Access tuple items using index — same as lists:

```
thistuple = ("apple", "banana", "cherry")

print(thistuple[1]) # banana

print(thistuple[-1]) # cherry

print(thistuple[0:2]) # ('apple', 'banana')
```

Tuple is Immutable — But You Can Workaround

You cannot change tuple values directly. However, you can convert it to a list, make changes, and convert it back to a tuple:

```
x = ("apple", "banana", "cherry")

y = list(x) # convert to list

y[1] = "kiwi" # make change

x = tuple(y) # convert back to tuple

print(x) # ('apple', 'kiwi', 'cherry')
```

Tuple Unpacking

Unpacking means extracting tuple values into individual variables. The number of variables must match the number of items in the tuple (unless you use an asterisk *).

```
fruits = ("apple", "banana", "cherry")

(green, yellow, red) = fruits

print(green) # apple

print(yellow) # banana

print(red) # cherry


# Using * to capture remaining items into a list

(first, *middle, last) = ("a", "b", "c", "d", "e")

print(first) # a

print(middle) # ['b', 'c', 'd']

print(last) # e
```

Tuple Operations

```
# Joining tuples

tuple1 = ("a", "b", "c")

tuple2 = (1, 2, 3)

tuple3 = tuple1 + tuple2
```



```
print(tuple3) # ('a', 'b', 'c', 1, 2, 3)

# Repeating tuples

fruits = ("apple", "banana")

print(fruits * 2) # ('apple', 'banana', 'apple', 'banana')

# Check if item exists

print("banana" in fruits) # True
```

Chapter 13: Python Sets

What is a Set?

A set is an unordered, unindexed collection of unique items. Sets are defined using curly braces {} or the set() constructor. Because sets are unordered, items have no index — you cannot access them by position. Sets automatically remove duplicate values, making them ideal for storing unique items and performing mathematical set operations like union, intersection, and difference.

```
# Creating a set

thisset = {"apple", "banana", "cherry"}

print(thisset) # order is random!


# Duplicates are automatically removed

thisset = {"apple", "banana", "cherry", "apple"}

print(thisset) # {'apple', 'banana', 'cherry'} – only 3 items


print(len(thisset)) # 3


print(type(thisset)) # <class 'set'>
```

■ *Note: Sets are unordered — the order of items may be different every time you print. You cannot use an index to access items.*

Adding and Removing Items

```
thisset = {"apple", "banana", "cherry"}


# add() – adds ONE item

thisset.add("orange")

print(thisset)


# update() – adds items from another set/list/tuple

thisset.update(["mango", "grapes"])

print(thisset)
```

```
# remove() – removes specific item (raises KeyError if not found)

thisset.remove("banana")

# discard() – removes specific item (NO error if not found)

thisset.discard("banana") # no error even if banana is gone

# pop() – removes a RANDOM item (returns it)

x = thisset.pop()

print(x) # some random item
```

Set Operations (Mathematical)

```
set1 = {"apple", "banana", "cherry"}

set2 = {"google", "microsoft", "apple"}

# union() – all items from both sets (no duplicates)

print(set1.union(set2)) # or: set1 | set2

# intersection() – only items in BOTH sets

print(set1.intersection(set2)) # or: set1 & set2

# difference() – items in set1 but NOT in set2

print(set1.difference(set2)) # or: set1 - set2

# symmetric_difference() – items in EITHER but not BOTH

print(set1.symmetric_difference(set2)) # or: set1 ^ set2
```

Output:

```
{'apple', 'banana', 'cherry', 'google', 'microsoft'}

{'apple'}

{'banana', 'cherry'}
```

```
{'banana', 'cherry', 'google', 'microsoft'}
```

Chapter 14: Python Dictionaries

What is a Dictionary?

A dictionary is an ordered (Python 3.7+), mutable collection of key-value pairs. Dictionaries are defined using curly braces {} with each item being a key:value pair separated by colons. Keys must be unique and immutable (strings, numbers, tuples), while values can be of any type. Dictionaries are also called associative arrays or hash maps in other languages.

```
# Creating a dictionary

thisdict = {

    "brand": "Ford",

    "model": "Mustang",

    "year": 1964,

    "colors": ["red", "white"] # value can be a list

}

print(thisdict)

print(len(thisdict)) # 4
```

Accessing Dictionary Values

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Method 1: Using square brackets

print(thisdict["model"]) # Mustang

# Method 2: Using get() – safer (no error if key missing)

print(thisdict.get("model")) # Mustang

print(thisdict.get("color")) # None (key doesn't exist)

print(thisdict.get("color", "Unknown")) # Unknown (default value)

# Get all keys
```

```
print(thisdict.keys()) # dict_keys(['brand', 'model', 'year'])

# Get all values

print(thisdict.values()) # dict_values(['Ford', 'Mustang', 1964])

# Get all key-value pairs

print(thisdict.items()) # dict_items([('brand', 'Ford')...])
```

Adding, Changing, and Removing Items

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Change a value

thisdict["year"] = 2020

# Or use update()

thisdict.update({"year": 2020})

# Add a new key-value pair

thisdict["color"] = "red"

# Remove with pop() – returns the removed value

thisdict.pop("model")

# Remove with del

del thisdict["color"]

# popitem() – removes last inserted item

thisdict.popitem()

# clear() – empties the dictionary

thisdict.clear()
```

Looping Through a Dictionary

```
thisdict = {"brand": "Ford", "model": "Mustang", "year": 1964}

# Loop through keys (default)

for x in thisdict:

    print(x) # brand, model, year

# Loop through values

for x in thisdict.values():

    print(x) # Ford, Mustang, 1964

# Loop through key-value pairs

for key, value in thisdict.items():

    print(key, "->", value)
```

Output:

```
brand -> Ford
model -> Mustang
year -> 1964
```

Nested Dictionaries

```
myfamily = {

    "child1": {"name": "Emil", "year": 2004},

    "child2": {"name": "Tobias", "year": 2007},

    "child3": {"name": "Linus", "year": 2011},

}

# Accessing nested values

print(myfamily["child2"]["name"]) # Tobias
```

Chapter 15: Python If...Else (Conditions)

What are Conditional Statements?

Conditional statements allow your program to make decisions and execute different blocks of code based on whether certain conditions are True or False. This is the fundamental concept of control flow in programming. Python uses if, elif (else if), and else keywords for conditional logic.

The if Statement

The if statement evaluates a condition. If the condition is True, the indented block of code under it is executed. If the condition is False, the block is skipped.

```
a = 33

b = 200

if b > a:

    print("b is greater than a")

    print("This also runs if condition is True")

# Condition is False – this block is skipped

if a > b:

    print("This will NOT print")

print("This always prints")
```

Output:

```
b is greater than a
This also runs if condition is True
This always prints
```

The elif Statement

The elif statement allows you to check additional conditions if the first if condition was False. You can have as many elif statements as you need. elif is short for 'else if'.

```
a = 33
```



```
b = 33

if b > a:

    print("b is greater than a")

elif a == b:

    print("a and b are equal")

elif a > b:

    print("a is greater than b")
```

Output:

```
a and b are equal
```

The else Statement

The else block catches everything that isn't caught by the preceding if and elif conditions. It runs when none of the above conditions are True. The else block has no condition.

```
a = 200

b = 33

if b > a:

    print("b is greater than a")

elif a == b:

    print("a and b are equal")

else:

    print("a is greater than b") # This runs
```

Output:

```
a is greater than b
```

Short-Hand If (Ternary Operator)

Python supports a one-line if-else syntax called the ternary operator or conditional expression. Syntax: value_if_true if condition else value_if_false

```
a = 2

b = 330

# Long form

if a > b:

    print("A")

else:

    print("B")

# Short form – same result

print("A" if a > b else print("B")) # B

# Assign using ternary

result = "A" if a > b else "B"

print(result) # B
```

Output:

B
B
B

Nested If Statements

You can put if statements inside other if statements — these are called nested if statements:

```
x = 41

if x > 10:

    print("Above ten,")

    if x > 20:

        print("and also above 20!")

    if x > 30:
```

```
print("and even above 30!")  
  
else:  
  
print("but not above 20.")
```

Output:

```
Above ten,  
and also above 20!  
and even above 30!
```

■ *Note: Python does not have a switch statement (before Python 3.10). Use if/elif/else chains instead. Python 3.10+ introduced match/case as an alternative.*

Chapter 16: Python While Loops

What is a While Loop?

A while loop repeatedly executes a block of code as long as a specified condition is True. The loop checks the condition before each iteration. If the condition is False at the very start, the loop body never executes. While loops are used when you don't know in advance how many times the loop needs to run.

■ *Important: Be careful with while loops! If the condition never becomes False, you will create an infinite loop that runs forever and freezes your program. Always make sure the loop variable is updated inside the loop.*

Basic While Loop

```
i = 1

while i < 6:

    print(i) # prints 1, 2, 3, 4, 5

    i += 1 # IMPORTANT: update i or loop runs forever!

print("Loop finished!")
```

Output:

```
1
2
3
4
5

Loop finished!
```

The break Statement

The break statement immediately exits the loop, regardless of the loop condition. It is used to stop the loop early when a certain condition is met:

```
i = 1

while i < 6:
```

```
print(i)

if i == 3:

    break # exit the loop when i equals 3

i += 1

# Output: 1, 2, 3 (stops at 3, does not reach 4, 5)
```

Output:

```
1
2
3
```

The continue Statement

The continue statement skips the rest of the code in the current iteration and jumps back to check the loop condition again — effectively skipping specific iterations:

```
i = 0

while i < 6:

    i += 1

    if i == 3:

        continue # skip printing when i == 3

    print(i)

# Output: 1, 2, 4, 5, 6 (3 is skipped)
```

Output:

```
1
2
4
5
6
```

The else Clause in While Loops

Python uniquely allows an else block at the end of a while loop. The else block executes when the loop condition becomes False naturally. If the loop is terminated by a break statement, the else block is NOT executed.

```
i = 1

while i < 6:

    print(i)

    i += 1

else:

    print("i is no longer less than 6 – loop ended normally")


# With break – else does NOT run:

i = 1

while i < 6:

    print(i)

    if i == 3:

        break

    i += 1

else:

    print("This will NOT print because break was used")
```

Chapter 17: Python For Loops

What is a For Loop?

A for loop is used to iterate over a sequence (such as a list, tuple, set, dictionary, string, or range) and execute a block of code for each item. Unlike while loops where you manually manage the loop variable, the for loop automatically handles iteration. For loops are ideal when you know exactly what you want to iterate over.

Basic For Loop

```
# Iterate over a list

fruits = ["apple", "banana", "cherry"]

for x in fruits:

    print(x)


# Iterate over a string – character by character

for x in "banana":

    print(x)


# Iterate over a tuple

for x in (1, 2, 3, 4, 5):

    print(x, end=' ') # end=' ' prints on same line
```

Output:

```
apple
banana
cherry

b
a
n
a
n
```

The range() Function

The `range()` function generates a sequence of numbers and is commonly used with `for` loops to execute code a specific number of times. Syntax: `range(start, stop, step)` — `stop` is exclusive (not included).

```
# range(stop) – from 0 to stop-1

for x in range(6): # 0, 1, 2, 3, 4, 5

    print(x, end=' ')

print() # newline


# range(start, stop) – from start to stop-1

for x in range(2, 6): # 2, 3, 4, 5

    print(x, end=' ')

print()


# range(start, stop, step) – with step value

for x in range(2, 30, 3): # 2, 5, 8, 11, ...

    print(x, end=' ')

print()


# Counting backwards with negative step

for x in range(10, 0, -1): # 10, 9, 8, ..., 1

    print(x, end=' ')
```

enumerate() — Loop with Index

When you need both the index and the value while iterating, use `enumerate()`. It adds a counter to an iterable and returns it as an `enumerate` object.

```
fruits = ["apple", "banana", "cherry"]
```



```
for i, fruit in enumerate(fruits):  
  
    print(f"Index {i}: {fruit}")  
  
# Custom start index  
  
for i, fruit in enumerate(fruits, start=1):  
  
    print(f"{i}. {fruit}")
```

Output:

```
Index 0: apple  
  
Index 1: banana  
  
Index 2: cherry  
  
1. apple  
  
2. banana  
  
3. cherry
```

Nested For Loops

You can put a for loop inside another for loop. The inner loop will execute fully for each iteration of the outer loop. Nested loops are used for working with multi-dimensional data like matrices.

```
adj = ["red", "big", "tasty"]  
  
fruits = ["apple", "banana", "cherry"]  
  
for x in adj:  
  
    for y in fruits:  
  
        print(x, y)  
  
# Output: red apple, red banana, red cherry,  
  
# big apple, big banana, big cherry,  
  
# tasty apple, tasty banana, tasty cherry
```

break, continue, else in For Loops

```
# break – stop the loop early
```

```
fruits = ["apple", "banana", "cherry"]

for x in fruits:
    if x == "banana":
        break

    print(x) # only prints 'apple'

# continue – skip current iteration

for x in fruits:
    if x == "banana":
        continue

    print(x) # prints 'apple' and 'cherry'

# else – runs after loop completes (not after break)

for x in range(3):
    print(x)
else:
    print("Loop completed!")
```

Chapter 18: Python Functions

What is a Function?

A function is a reusable block of code that performs a specific task. Functions are defined once and can be called (executed) multiple times from anywhere in the program. Functions help avoid code duplication (DRY — Don't Repeat Yourself), make code more organized, easier to read, test, and debug. In Python, functions are defined using the `def` keyword.

There are two types of functions in Python: Built-in functions (provided by Python, like `print()`, `len()`, `type()`, `range()`) and User-defined functions (functions you create yourself).

Creating and Calling a Function

```
# Define a function

def my_function():

    print("Hello from a function!")

    print("This code runs when the function is called")


# Call (invoke) the function

my_function() # executes the function body

my_function() # you can call it multiple times
```

Output:

```
Hello from a function!

This code runs when the function is called

Hello from a function!

This code runs when the function is called
```

Function Parameters and Arguments

Parameters are variables listed in the function definition (inside the parentheses). Arguments are the actual values you pass to the function when calling it.

```
# fname is the parameter

def my_function(fname):

    print(fname + " Refsnes")
```

```
# 'Emil', 'Tobias', 'Linus' are arguments

my_function("Emil") # Emil Refsnes

my_function("Tobias") # Tobias Refsnes

my_function("Linus") # Linus Refsnes


# Multiple parameters

def greet(fname, lname):

    print("Hello", fname, lname)


greet("John", "Doe")
```

Default Parameter Values

You can set a default value for a parameter. If no argument is passed for that parameter, the default value is used. Default parameters must come after required parameters.

```
def my_function(country="Norway"):

    print("I am from " + country)


my_function("Sweden") # I am from Sweden

my_function("India") # I am from India

my_function() # I am from Norway (default)
```

Arbitrary Arguments — *args

If you don't know how many arguments will be passed, use *args. The function receives them as a tuple. The asterisk (*) tells Python to pack all the arguments into a tuple.

```
def my_function(*kids):

    print("The youngest child is " + kids[2])


my_function("Emil", "Tobias", "Linus") # Linus


# Sum of any number of values
```

```
def total(*nums):  
  
    print(sum(nums))  
  
total(1, 2, 3) # 6  
  
total(10, 20, 30, 40) # 100
```

Keyword Arguments — **kwargs

****kwargs** allows you to pass named (keyword) arguments. The function receives them as a dictionary. This is useful when the function needs to handle flexible named inputs.

```
def my_function(**kid):  
  
    print("His last name is " + kid["lname"])  
  
my_function(fname="Tobias", lname="Refsnes") # Refsnes  
  
# Print all keyword arguments  
  
def show_info(**info):  
  
    for key, value in info.items():  
  
        print(f"{key}: {value}")  
  
show_info(name="Alice", age=30, city="London")
```

Return Values

A function can return a value using the return statement. When Python encounters return, it immediately exits the function and sends the value back to the caller. A function can return any data type, including lists, tuples, and dictionaries.

```
def my_function(x):  
  
    return 5 * x  
  
print(my_function(3)) # 15  
  
print(my_function(5)) # 25  
  
print(my_function(9)) # 45
```

```
# Return multiple values (as a tuple)

def min_max(numbers):

    return min(numbers), max(numbers)

low, high = min_max([3, 1, 4, 1, 5, 9, 2, 6])

print(low, high) # 1 9
```

Output:

```
15
25
45
1 9
```

Recursion

A recursive function is a function that calls itself. Every recursive function must have a base case (a condition that stops the recursion) and a recursive case (where the function calls itself). Without a base case, the function would call itself infinitely.

```
# Classic example: factorial

def factorial(n):

    if n == 1: # base case – stops recursion

        return 1

    else:

        return n * factorial(n - 1) # recursive call

print(factorial(5)) # 5 * 4 * 3 * 2 * 1 = 120

print(factorial(3)) # 3 * 2 * 1 = 6
```

Output:

```
120
6
```

Chapter 19: Python Lambda Functions

What is a Lambda Function?

A lambda function is a small, anonymous (unnamed) function defined using the lambda keyword. Unlike regular functions defined with def, lambda functions can have any number of input arguments but can only have ONE expression. The expression is automatically returned — you don't write return. Lambda functions are sometimes called 'arrow functions' or 'anonymous functions' in other languages.

Syntax: lambda arguments : expression

Lambda vs Regular Function

```
# Regular function

def add(a, b):

    return a + b

print(add(5, 3)) # 8


# Equivalent lambda function

add = lambda a, b: a + b

print(add(5, 3)) # 8


# Lambda with one argument

double = lambda x: x * 2

print(double(5)) # 10


# Lambda with three arguments

multiply = lambda a, b, c: a * b * c

print(multiply(2, 3, 4)) # 24
```

Output:

8

8

10

24

Why Use Lambda? — Use with Higher-Order Functions

Lambda functions are most useful when you need a simple function for a short period of time, typically when passing it as an argument to another function. The most common use cases are with `map()`, `filter()`, and `sorted()`:

```
# map() – applies a function to every item in a list

numbers = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x**2, numbers))

print(squared) # [1, 4, 9, 16, 25]


# filter() – filters items based on a condition

evens = list(filter(lambda x: x % 2 == 0, numbers))

print(evens) # [2, 4]


# sorted() with custom key

words = ["banana", "apple", "cherry", "date"]

sorted_words = sorted(words, key=lambda x: len(x))

print(sorted_words) # ['date', 'apple', 'banana', 'cherry']
```

Output:

```
[1, 4, 9, 16, 25]

[2, 4]

['date', 'apple', 'banana', 'cherry']
```

Lambda Inside a Function — Factory Pattern

Lambda functions are often used as function factories — functions that return other functions configured with specific behaviors:

```
def multiplier(n):

    return lambda x: x * n # returns a lambda
```



```
double = multiplier(2)

triple = multiplier(3)

tenTimes = multiplier(10)

print(double(5)) # 10

print(triple(5)) # 15

print(tenTimes(5)) # 50
```

Output:

10

15

50

Chapter 20: Python Arrays

Arrays in Python

Python does not have a built-in array data type in the same way as C, C++, or Java. However, Python lists can be used as arrays. The key difference between Python lists and arrays in other languages is that Python lists can contain items of different data types, while traditional arrays hold items of the same type.

For performance-critical applications that deal with large amounts of numerical data, Python developers use the NumPy library which provides a proper array object (ndarray) that is much faster and more memory-efficient than Python lists.

Using Lists as Arrays

```
# Array of car brands

cars = ["Ford", "Volvo", "BMW"]


# Access an array element

print(cars[0]) # Ford


# Modify an element

cars[0] = "Toyota"

print(cars) # ['Toyota', 'Volvo', 'BMW']


# Length of array

print(len(cars)) # 3


# Loop through array

for x in cars:

    print(x)
```

Common Array Operations

```
cars = ["Ford", "Volvo", "BMW"]
```

```
# Add element at end

cars.append("Honda")

print(cars) # ['Ford', 'Volvo', 'BMW', 'Honda']


# Remove element

cars.remove("Volvo")

print(cars) # ['Ford', 'BMW', 'Honda']


# Insert at specific position

cars.insert(1, "Tesla")

print(cars) # ['Ford', 'Tesla', 'BMW', 'Honda']


# Sort array

cars.sort()

print(cars) # ['BMW', 'Ford', 'Honda', 'Tesla']
```

NumPy Arrays — For Numerical Computing

NumPy (Numerical Python) is the most popular library for working with arrays in Python. NumPy arrays are faster, use less memory, and support vectorized operations (performing operations on entire arrays at once without looping).

```
import numpy as np


# Create a 1D array

arr = np.array([1, 2, 3, 4, 5])

print(arr) # [1 2 3 4 5]

print(type(arr)) # <class 'numpy.ndarray'>


# Create a 2D array (matrix)

matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(matrix)
```

```
# Array operations (element-wise)

a = np.array([1, 2, 3])

b = np.array([4, 5, 6])

print(a + b) # [5 7 9]

print(a * b) # [4 10 18]

print(a * 2) # [2 4 6] – scalar multiplication
```

Chapter 21: Python Classes & Objects

Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around objects rather than functions and logic. An object is a self-contained unit that combines data (attributes/properties) and behavior (methods/functions). Python is an object-oriented language — almost everything is an object, including integers, strings, and lists.

A class is like a blueprint or template for creating objects. Just as you can build many houses from the same architectural blueprint, you can create many objects from the same class. The objects created from a class are called instances.

Creating a Class

```
# Define a class using the 'class' keyword

class MyClass:

    x = 5 # class attribute


# Create an object (instance) from the class

p1 = MyClass()

print(p1.x) # 5
```

The __init__() Method (Constructor)

The __init__() method is a special method (called a dunder or magic method) that is automatically called when you create a new object. It is used to initialize the object's attributes. The first parameter of every method in a class is self, which refers to the current instance of the class.

```
class Person:

    def __init__(self, name, age):

        self.name = name # instance attribute

        self.age = age # instance attribute


    def greet(self):

        print(f"Hello, my name is {self.name} and I am {self.age} years old.")
```

```
# Creating objects

p1 = Person("John", 36)

p2 = Person("Alice", 25)


print(p1.name) # John

print(p1.age) # 36

p1.greet() # Hello, my name is John and I am 36 years old.

p2.greet() # Hello, my name is Alice and I am 25 years old.
```

Output:

```
John

36

Hello, my name is John and I am 36 years old.

Hello, my name is Alice and I am 25 years old.
```

Class Methods

Methods are functions defined inside a class that define the behaviors of objects. They always have self as the first parameter.

```
class BankAccount:

    def __init__(self, owner, balance=0):

        self.owner = owner

        self.balance = balance


    def deposit(self, amount):

        self.balance += amount

        print(f"Deposited {amount}. Balance: {self.balance}")


    def withdraw(self, amount):

        if amount <= self.balance:

            self.balance -= amount
```

```
print(f"Withdrew {amount}. Balance: {self.balance}")

else:

print("Insufficient funds!")


def __str__(self): # called by print()

return f"Account({self.owner}, Balance: {self.balance})"


acc = BankAccount("Bob", 1000)

acc.deposit(500)

acc.withdraw(200)

print(acc)
```

Output:

```
Deposited 500. Balance: 1500

Withdrew 200. Balance: 1300

Account(Bob, Balance: 1300)
```

Modifying and Deleting Object Properties

```
p1 = Person("John", 36)


# Modify property

p1.age = 40

print(p1.age) # 40


# Delete a property

del p1.age

# print(p1.age) # would raise AttributeError


# Delete an object

del p1
```

```
# print(p1) # would raise NameError
```


Chapter 22: Python Inheritance

What is Inheritance?

Inheritance is a fundamental OOP concept that allows one class (the child or subclass) to inherit all the attributes and methods of another class (the parent or base class). This promotes code reuse — you don't have to rewrite code that already exists in a parent class. The child class can also add new attributes and methods or override (modify) existing ones.

The syntax for inheritance is: `class ChildClass(ParentClass):`

Parent Class

```
class Person:

    def __init__(self, fname, lname):

        self.firstname = fname

        self.lastname = lname

    def printname(self):

        print(self.firstname, self.lastname)

# Using the parent class

x = Person("John", "Doe")

x.printname() # John Doe
```

Simple Child Class — using pass

```
# Student inherits everything from Person

class Student(Person):

    pass # no new content, just inherits everything

x = Student("Mike", "Olsen")

x.printname() # Mike Olsen – inherited method works!
```

Child Class with `__init__()`

When you add an `__init__()` in the child class, it overrides the parent's `__init__()`. To keep the parent's initialization, call `super().__init__()` inside the child's `__init__()`.

```
class Student(Person):

    def __init__(self, fname, lname, year):

        super().__init__(fname, lname) # call parent's __init__

        self.graduationyear = year # add new attribute

    def welcome(self):

        print(f"Welcome {self.firstname} {self.lastname}, "

              f" class of {self.graduationyear}")

x = Student("Mike", "Olsen", 2023)

x.printname() # Mike Olsen (inherited method)

x.welcome() # Welcome Mike Olsen, class of 2023

print(x.graduationyear) # 2023
```

Output:

```
Mike Olsen

Welcome Mike Olsen, class of 2023

2023
```

Method Overriding

A child class can provide a different implementation of a method inherited from the parent:

```
class Animal:

    def speak(self):

        print("The animal makes a sound")

class Dog(Animal):

    def speak(self): # Override parent method

        print("The dog barks: Woof!")
```

```
class Cat(Animal):  
  
    def speak(self): # Override parent method  
  
        print("The cat meows: Meow!")  
  
a = Animal()  
  
d = Dog()  
  
c = Cat()  
  
a.speak() # The animal makes a sound  
  
d.speak() # The dog barks: Woof!  
  
c.speak() # The cat meows: Meow!
```

Chapter 23: Python Iterators

What is an Iterator?

An iterator is an object that contains a countable number of values and can be traversed (iterated) one value at a time. In Python, an iterator implements two methods: `__iter__()` which returns the iterator object itself, and `__next__()` which returns the next value in the sequence.

All iterables (lists, tuples, strings, sets, dictionaries) have an `__iter__()` method that returns an iterator. When you use a for loop, Python automatically creates an iterator and calls `__next__()` behind the scenes.

Using Iterators

```
# Creating an iterator from a list

mytuple = ('apple', 'banana', 'cherry')

myit = iter(mytuple)

print(next(myit)) # apple

print(next(myit)) # banana

print(next(myit)) # cherry

# print(next(myit)) # StopIteration error – no more items


# Strings are iterable

mystr = 'banana'

myit2 = iter(mystr)

print(next(myit2)) # b

print(next(myit2)) # a
```

Output:

```
apple
banana
cherry
b
```

Creating a Custom Iterator

```
class NumberRange:

    def __init__(self, start, end):

        self.current = start

        self.end = end

    def __iter__(self):

        return self

    def __next__(self):

        if self.current > self.end:

            raise StopIteration

        value = self.current

        self.current += 1

        return value

# Use the custom iterator

myrange = NumberRange(1, 5)

for num in myrange:

    print(num, end=' ') # 1 2 3 4 5
```

Chapter 24: Python Scope

What is Scope?

Scope refers to the region of the program where a variable is recognized and accessible. Python follows the LEGB rule to determine variable scope — Local, Enclosing, Global, Built-in. Python searches for a variable in this order: first locally, then in any enclosing functions, then globally, and finally in the built-in namespace.

Local Scope

A variable created inside a function belongs to the local scope of that function and can only be accessed inside that function:

```
def my_function():  
  
    x = 300 # local variable  
  
    print(x) # works fine  
  
my_function() # 300  
  
# print(x) # NameError: x is not defined outside the function
```

Global Scope

A variable created in the main body of Python code is a global variable and belongs to the global scope. It is accessible from anywhere:

```
x = 300 # global variable  
  
def my_function():  
  
    print(x) # can access global variable  
  
my_function() # 300  
  
print(x) # 300
```

The global Keyword

To create or modify a global variable from inside a function, use the global keyword:

```
x = 300
```

```
def my_function():  
  
    global x # tell Python to use the global x  
  
    x = 200 # now modifies the global x  
  
print(x) # 300 (before function call)  
  
my_function()  
  
print(x) # 200 (after function modified it)
```

Output:

```
300  
  
200
```

Nested Scopes and the nonlocal Keyword

```
def outer():  
  
    x = 10 # enclosing scope  
  
    def inner():  
  
        nonlocal x # modify the enclosing variable  
  
        x = 20  
  
        print('inner x:', x) # 20  
  
    inner()  
  
    print('outer x:', x) # 20 (modified by inner)  
  
outer()
```

Chapter 25: Python Modules

What is a Module?

A module is a file containing Python definitions and statements (functions, classes, variables). The filename is the module name with the .py extension. Modules allow you to organize your Python code logically into separate files. When a program gets large, you can split it into multiple modules for better organization and reusability. Any Python file can be used as a module.

Creating and Using a Module

```
# Save this as mymodule.py

def greeting(name):

    print("Hello, " + name)

person1 = {"name": "John", "age": 36, "country": "Norway"}
```

```
# In your main file (e.g., main.py)

import mymodule

mymodule.greeting("Jonathan") # Hello, Jonathan

mymodule.greeting("Alice") # Hello, Alice

# Accessing variables from the module

a = mymodule.person1["age"]

print(a) # 36
```

Output:

```
Hello, Jonathan

Hello, Alice

36
```

Import with Alias (as)

Use an alias to give a module a shorter name for convenience:


```
import mymodule as mx

mx.greeting("Alice") # Hello, Alice


import numpy as np # Very common in data science

import pandas as pd # Conventional alias

import matplotlib.pyplot as plt
```

from...import — Import Specific Parts

Instead of importing the entire module, you can import only what you need:

```
from mymodule import greeting, person1


# Now use directly without 'mymodule.' prefix

greeting("Alice") # Hello, Alice

print(person1["age"]) # 36


# Import everything (not recommended – can cause name conflicts)

from mymodule import *
```

Python's Standard Library — Built-in Modules

Python comes with a rich standard library of modules. Some important ones:

```
import math

print(math.pi) # 3.141592653589793

print(math.sqrt(64)) # 8.0

print(math.floor(4.7)) # 4

print(math.ceil(4.1)) # 5

print(math.pow(2, 10)) # 1024.0


import random

print(random.randint(1, 100)) # random integer
```

```
import os

print(os.getcwd()) # current directory

print(os.listdir('.')) # list directory contents


import sys

print(sys.version) # Python version


import datetime

print(datetime.datetime.now()) # current date and time
```

Chapter 26: Python Dates

Python datetime Module

Python does not have a built-in date data type. To work with dates and times, you use the datetime module from the standard library. The datetime module provides classes for working with dates, times, and time intervals.

```
import datetime

# Current date and time

x = datetime.datetime.now()

print(x) # e.g., 2024-01-15 14:30:25.123456

print(type(x)) # <class 'datetime.datetime'>

# Accessing individual components

print(x.year) # 2024

print(x.month) # 1

print(x.day) # 15

print(x.hour) # 14

print(x.minute) # 30

print(x.second) # 25
```

Creating Date Objects

```
import datetime

# Create a specific date: datetime(year, month, day, hour, minute, second)

x = datetime.datetime(2020, 5, 17)

print(x) # 2020-05-17 00:00:00

# Date only
```

```
d = datetime.date(2024, 1, 15)

print(d) # 2024-01-15

# Today's date

today = datetime.date.today()

print(today)
```

Formatting Dates — strftime()

strftime() formats a datetime object into a readable string. The format codes are:

Code	Description	Example
%Y	Year — 4 digits	2024
%y	Year — 2 digits	24
%m	Month — zero-padded	01
%B	Month — full name	January
%b	Month — abbreviated	Jan
%d	Day — zero-padded	15
%A	Weekday — full name	Monday
%a	Weekday — abbreviated	Mon
%H	Hour — 24hr format	14
%I	Hour — 12hr format	02
%M	Minute	30
%S	Second	25
%p	AM/PM	PM
%f	Microseconds	123456

```
import datetime

x = datetime.datetime(2024, 6, 15, 14, 30, 0)

print(x.strftime("%B %d, %Y")) # June 15, 2024

print(x.strftime("%d/%m/%Y %H:%M:%S")) # 15/06/2024 14:30:00

print(x.strftime("%A, %B %d, %Y")) # Saturday, June 15, 2024
```

Chapter 27: Python File Handling

File Handling Overview

File handling allows your programs to create, read, update, and delete files stored on disk. This is essential for working with data that persists between program runs. Python provides a built-in `open()` function to work with files. The file modes are:

Mode	Full Name	Description
r	Read	Opens file for reading. Raises <code>FileNotFoundError</code> if file doesn't exist. (Default mode)
w	Write	Opens file for writing. Creates file if it doesn't exist. OVERWRITES existing content.
a	Append	Opens file for appending. Creates file if it doesn't exist. Adds to end of file.
x	Create	Creates a new file. Raises <code>FileExistsError</code> if file already exists.
r+	Read/Write	Opens file for both reading and writing. File must exist.
b	Binary	Add to any mode (e.g., 'rb') to open in binary mode for images, videos, etc.

Opening and Reading Files

```
# Basic file read

f = open("demofile.txt", "r")

print(f.read()) # reads the entire file

f.close() # always close the file when done!


# Read only the first 5 characters

f = open("demofile.txt", "r")

print(f.read(5))

f.close()


# Read one line at a time

f = open("demofile.txt", "r")

print(f.readline()) # first line

print(f.readline()) # second line
```

```
f.close()

# Loop through all lines

f = open("demofile.txt", "r")

for line in f:

    print(line, end='') # print each line

f.close()
```

Using with Statement (Best Practice)

The with statement automatically closes the file when the block finishes, even if an error occurs. This is the recommended way to handle files in Python — you never have to manually call `f.close()`.

```
# Reading with 'with' – file is automatically closed

with open("demofile.txt", "r") as f:

    content = f.read()

    print(content)

# File is now closed automatically

# Reading all lines into a list

with open("demofile.txt", "r") as f:

    lines = f.readlines() # returns list of lines

    for line in lines:

        print(line.strip()) # strip() removes newline
```

Writing to Files

```
# 'w' mode – creates file or OVERWRITES existing content

with open("demofile2.txt", "w") as f:

    f.write("Hello! I have created a new file\n")

    f.write("And written to it!")
```

```
# 'a' mode – appends to existing file

with open("demofile2.txt", "a") as f:

    f.write("\nThis line is appended!")


# Writing multiple lines

lines = ["Line 1\n", "Line 2\n", "Line 3\n"]

with open("output.txt", "w") as f:

    f.writelines(lines)
```

Creating and Deleting Files

```
import os


# Create a new file (raises error if file exists)

with open("newfile.txt", "x") as f:

    f.write("This is a new file")


# Delete a file

os.remove("demofile.txt")


# Check if file exists before deleting

if os.path.exists("demofile.txt"):

    os.remove("demofile.txt")

else:

    print("The file does not exist")


# Delete an empty folder

os.rmdir("myfolder")


# Check if file/directory exists
```

```
print(os.path.exists("myfile.txt")) # True or False  
  
print(os.path.isfile("myfile.txt")) # True if it is a file  
  
print(os.path.isdir("myfolder")) # True if it is a directory
```


Chapter 28: Python Exception Handling

What are Exceptions?

An exception is an error that occurs during the execution of a program. When Python encounters an error, it normally stops and generates an error message. Exception handling allows you to gracefully handle these errors instead of crashing the program. This makes your programs more robust and user-friendly.

Python uses try, except, else, and finally blocks for exception handling. The basic idea is: try to run some code, and if it fails (raises an exception), catch it and handle it gracefully.

try and except

The try block contains the code that might raise an exception. The except block contains the code to execute if an exception occurs:

```
try:

    print(x) # x is not defined – will raise NameError

except:

    print("An exception occurred") # this runs instead of crashing

print('Program continues...') # program keeps running
```

Output:

```
An exception occurred

Program continues...
```

Handling Specific Exceptions

You should specify the type of exception to catch specific errors. This is better practice because it prevents accidentally hiding unexpected errors:

```
try:

    x = int('abc') # ValueError: invalid literal for int()

except ValueError:

    print("That's not a valid number!")

except TypeError:
```

```
print("Wrong data type!")

except Exception as e:

    print(f"Unexpected error: {e}")

# Multiple exceptions in one except

try:

    result = 10 / 0

except (ZeroDivisionError, ValueError) as e:

    print(f"Error: {e}")
```

Output:

```
That's not a valid number!

Error: division by zero
```

The else Clause

The else block runs only if the try block did NOT raise any exception:

```
try:

    x = int("5") # this succeeds

except ValueError:

    print("Not a valid number")

else:

    print(f"Successfully converted: {x}") # this runs

    print("No errors occurred!")
```

Output:

```
Successfully converted: 5

No errors occurred!
```

The finally Clause

The finally block always executes regardless of whether an exception occurred or not. It is typically used for cleanup operations like closing files, releasing resources, or closing database connections.

```

try:

    f = open("myfile.txt", "r")

    content = f.read()

    result = int(content) # might fail if content is not a number

except FileNotFoundError:

    print("File not found!")

except ValueError:

    print("File content is not a number!")

else:

    print(f"File read successfully: {result}")

finally:

    print("This ALWAYS runs – cleanup code here")

    # close file if it was opened

try:

    f.close()

except:

    pass

```

Raising Exceptions

You can intentionally raise exceptions using the raise keyword to signal that an error has occurred:

```

def set_age(age):

    if not isinstance(age, int):

        raise TypeError("Age must be an integer")

    if age < 0 or age > 150:

        raise ValueError(f"Age {age} is not realistic")

    return age

```

```

try:

    set_age(-5)

except ValueError as e:

    print(f"ValueError: {e}")


try:

    set_age("twenty")

except TypeError as e:

    print(f"TypeError: {e}")

```

Output:

```

ValueError: Age -5 is not realistic

TypeError: Age must be an integer

```

Common Python Exceptions Reference

Exception	When It Occurs
ZeroDivisionError	Dividing a number by zero: 10 / 0
NameError	Using a variable that hasn't been defined: print(xyz)
TypeError	Wrong type for an operation: '2' + 2
ValueError	Correct type but invalid value: int('abc')
IndexError	List index is out of range: lst[100] on small list
KeyError	Dictionary key does not exist: d['missing_key']
AttributeError	Accessing non-existent attribute: 'hello'.nonexistent()
FileNotFoundError	Opening a file that doesn't exist
ImportError	Module cannot be found or imported
ModuleNotFoundError	Module is not installed
PermissionError	No permission to access a file
RecursionError	Maximum recursion depth exceeded
OverflowError	Arithmetic result too large to be represented
MemoryError	Operation ran out of memory
StopIteration	next() called on exhausted iterator

RuntimeError	General runtime error that doesn't fit other categories
NotImplementedError	Abstract method that must be overridden has not been
OSError / IOError	Operating system level error (file/network operations)
UnicodeDecodeError	Decoding error when reading non-UTF-8 encoded files

Chapter 29: Python Quick Reference Card

Syntax Cheat Sheet

Topic	Syntax / Example
Print output	<code>print("Hello, World!")</code>
Input from user	<code>name = input("Enter your name: ")</code>
Variable assign	<code>x = 5; name = "Alice"; pi = 3.14</code>
Multiple assign	<code>a, b, c = 1, 2, 3</code>
Type check	<code>type(x) → <class "int"></code>
Type casting	<code>int("5"), float("3.14"), str(42)</code>
String f-format	<code>f"Hello {name}, you are {age}"</code>
String methods	<code>"hello".upper(), " hi ".strip()</code>
List create	<code>lst = [1, 2, 3]; lst.append(4)</code>
List comprehension	<code>[x*2 for x in range(5) if x>1]</code>
Tuple create	<code>tpl = (1, 2, 3) # immutable</code>
Set create	<code>st = {1, 2, 3} # unique, unordered</code>
Dict create	<code>d = {"key": "val"}; d["key"]</code>
Dict loop	<code>for k, v in d.items(): print(k, v)</code>
If / elif / else	<code>if x>0: ... elif x==0: ... else: ...</code>
Ternary if	<code>"pos" if x>0 else "neg"</code>
While loop	<code>while x < 5: x += 1</code>
For loop	<code>for i in range(10): print(i)</code>
For + enumerate	<code>for i, v in enumerate(lst): ...</code>
Define function	<code>def add(a, b): return a + b</code>
Default param	<code>def greet(name="World"): ...</code>
*args **kwargs	<code>def f(*args, **kwargs): ...</code>
Lambda	<code>square = lambda x: x**2</code>
Map / Filter	<code>list(map(lambda x: x*2, lst))</code>
Class + init	<code>class Dog: def __init__(self, name): self.name=name</code>
Inheritance	<code>class Poodle(Dog): pass</code>

Import module	<code>import math; from os import path</code>
Try / Except	<code>try: ... except ValueError as e: print(e)</code>
Finally	<code>try: ... except: ... finally: cleanup()</code>
Open file read	<code>with open("f.txt") as f: data=f.read()</code>
Open file write	<code>with open("f.txt","w") as f: f.write("hi")</code>
List file to lines	<code>lines = f.readlines()</code>
Delete file	<code>import os; os.remove("file.txt")</code>

Operator Precedence (Highest to Lowest)

When multiple operators appear in an expression, Python follows this order:

Level	Operators	Description
1 (Highest)	<code>()</code>	Parentheses — always evaluated first
2	<code>**</code>	Exponentiation
3	<code>+x, -x, ~x</code>	Unary plus, minus, bitwise NOT
4	<code>*, /, //, %</code>	Multiplication, Division, Floor Div, Modulus
5	<code>+, -</code>	Addition, Subtraction
6	<code><<, >></code>	Bitwise shifts
7	<code>&</code>	Bitwise AND
8	<code>^</code>	Bitwise XOR
9	<code> </code>	Bitwise OR
10	<code>==, !=, <, >, <=, >=</code>	Comparison operators
11	<code>not</code>	Logical NOT
12	<code>and</code>	Logical AND
13 (Lowest)	<code>or</code>	Logical OR

End of Python Complete Notes — Based on W3Schools Python Tutorial — Happy Coding!